

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Thu 4 February 2016 - Part Theory

NAME:

MATRICOLA:

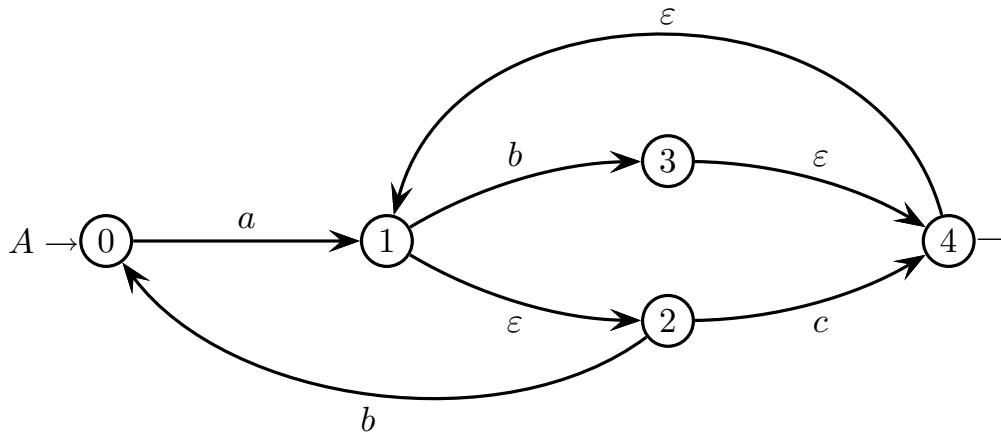
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the finite state automaton A shown in the figure below:



Answer the following questions:

- (a) In lexicographical order, write all the strings of length 3 accepted by automaton A , and for each of these strings show the computation(s) that accept(s) it.
 - (b) Write the right unilinear grammar equivalent to automaton A .
 - (c) Use the BS method to obtain a deterministic automaton A_d equivalent to A , but without any spontaneous moves.
 - (d) Adopting a systematic method, determine if the deterministic automaton A_d is minimal.
 - (e) Using a systematic method, obtain an automaton A_c that accepts the complement of the language accepted by A .
 - (f) (optional) Is automaton A ambiguous? Provide an adequate, possibly informal but not tautological, justification to your answer.
-

2 Free Grammars and Pushdown Automata 20%

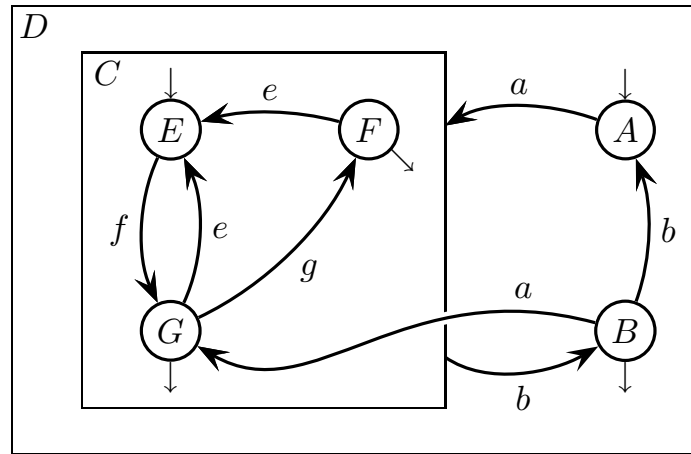
1. Consider the Dyck language with two parenthesis types, namely $a b$ (open closed) and $c d$ (open closed), i.e., over the alphabet $\Sigma = \{a, b, c, d\}$, generated by the well known non-ambiguous *BNF* grammar G shown below (axiom S):

$$G \left\{ \begin{array}{lcl} S & \rightarrow & a S b S \\ S & \rightarrow & c S d S \\ S & \rightarrow & \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Write a grammar G_1 , *BNF* and non-ambiguous, that generates language $L(G)$ minus all the Dyck strings that contain one or more pairs of adjacent letters $a c$.
 - (b) (optional) Write a grammar G_1 , *BNF* and non-ambiguous, that generates language $L(G)$ minus all the Dyck strings that contain one or more pairs of adjacent letters $b d$.
-

2. A state diagram, inspired to the notion of *Statecharts*, is a generalization of a finite state automaton, where any state may be refined into a lower-level state diagram. This refinement operation can be iterated at any depth level. The figure below shows the graphical representation of a state diagram D with three states, where state C is refined into a lower-level state diagram with states E , F and G .



The text description of the state diagram in the figure above is the following:

stateDiagram D is

```

state A, B, C;
initial A;
final B;
transition A to C label a, B to A label b;
transition C to B label b, B to C.G label a;

```

stateDiagram C is

```

state F, G, E;
initial E;
final G;
final F;
transition E to G label f, G to E label e, F to E label e,
G to F label g;

```

end stateDiagram C;

end stateDiagram D;

A state diagram must satisfy the following constraints:

- There is exactly one clause for declaring all the atomic states, i.e., the states not refined into a state (sub)diagram; notice that a diagram may include one or more atomic states, and that it may even be empty.
- There is exactly one initial state, declared in only one clause.
- There are one or more final states, declared in as many clauses (one per clause).
- A transition may connect a state of a diagram to a state at the same level or to one in a lower-level diagram, at any depth level, for instance as in:

transition Q to P.R.S label m

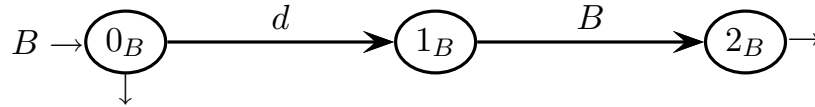
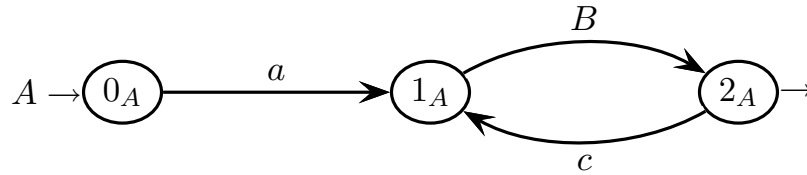
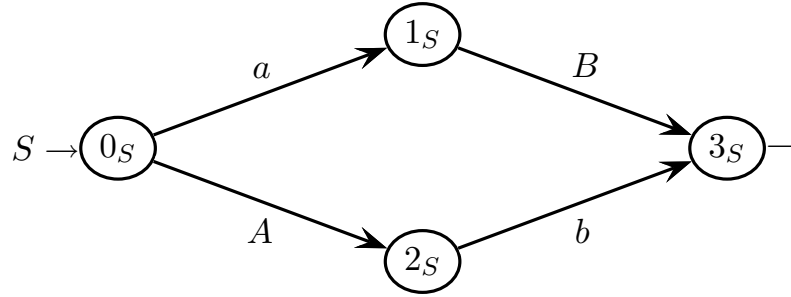
The syntax that generates the states must define the operator “.” to be left-associative, so that a state pathname like “P.R.S” is parsed as “(P.R).S”.

- The relative position of the various clauses within the definition of a diagram is defined by the provided example.

Write an extended (*EBNF*) grammar, not ambiguous, that generates a non-empty sequence of state diagram definitions according to the rules described above.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net (axiom S):



Answer the following questions:

- (a) Draw the complete *ELR* pilot graph of grammar G , say if grammar G is of type *ELR*(1) and justify your answer.
- (b) Find all the guide sets with $k = 1$ on the shift arcs, call arcs and exit arrows of the machine net of grammar G , say if grammar G is of type *ELL*(1) and justify your answer. Please use the net drawing above to annotate the guide sets.
- (c) In the case your answer to point (b) is negative, say if grammar G is of type *ELL*(k) for some $k > 1$ and shortly justify your answer.
- (d) (optional) In the case your answer to point (c) is negative, shortly say if there is a grammar G' that is equivalent to G and is of type *ELL*(k) for some $k \geq 1$, and shortly justify your answer.

4 Language Translation and Semantic Analysis 20%

1. Consider the translations τ_1 and τ_2 , defined respectively on the following source languages L_{s1} and L_{s2} :

$$L_{s1} = \{ a^n b^n c \mid n > 0 \} \cup \{ a^n b^n d \mid n > 0 \}$$

$$L_{s2} = \{ (ab)^n c \mid n > 0 \} \cup \{ (ab)^n d \mid n > 0 \}$$

Translations τ_1 and τ_2 work as follows, for any $n > 0$:

$$\begin{array}{ll} \tau_1 (a^n b^n c) &= c^n b^n a & \tau_2 ((ab)^n c) &= (cb)^n a \\ \tau_1 (a^n b^n d) &= a^n d^n b & \tau_2 ((ab)^n d) &= (ad)^n b \end{array}$$

Answer the following questions:

- (a) Define translation τ_1 by means of the simplest possible translation scheme, that is, either a regular translation expression or a translation grammar.
 - (b) Define translation τ_2 by means of the simplest possible translation scheme, that is, either a regular translation expression or a translation grammar.
 - (c) (optional) Determine if the translation τ_2 can be *computed* by some of the *deterministic* transducers, i.e., deterministic I/O automata, that have been studied in the course, and adequately justify your answer.
-

2. Consider the Dyck language with two parenthesis types, namely $a b$ (open closed) and $c d$ (open closed), generated by the known syntactic support below (axiom S):

$$\left\{ \begin{array}{lcl} 0: & S & \rightarrow D \\ 1: & D & \rightarrow a D b D \\ 2: & D & \rightarrow c D d D \\ 3: & D & \rightarrow \varepsilon \end{array} \right.$$

One wishes to analyze a syntax tree by means of attribute grammars, to check if the Dyck string contains certain pairs of adjacent letters, and to compute and output the nesting depth of the pair, if any, that occurs leftmost in the string, or to output 0 if there is no such pair. To answer the questions below, one has to use (partly or totally) the attributes already prepared on the next page.

Answer the following questions:

- (a) Find an attribute grammar G_1 that checks if the Dyck string contains an adjacent letter pair of type $a c$.
- (b) Find an attribute grammar G_2 that checks if the Dyck string contains an adjacent letter pair of type $b d$.
- (c) (optional) Reconsider grammar G_1 and extend it, in such a way that it outputs (at the tree root) the nesting depth (always ≥ 1) of the leftmost occurrence of the pair $a c$ (more precisely the depth of letter c in the pair), if the Dyck string has some, or else that outputs 0. Samples:
 - $a c d b$, yes (one pair), depth 1
 - $a b a c d b$, yes (one pair), depth 1
 - $a a b a c d b b$, yes (one pair), depth 2
 - $a c d a a c d b b$, yes (two pairs), depth 1
 - $a b c d$, none, depth 0

Furthermore, say if the so-extended attribute grammar is of type one-sweep and justify your answer. To answer question (c), you can (and actually have to) define more attributes, for dealing with the nesting depth. Extend the attribute list already given on the next page.

attributes assigned to be used (complete the specification if necessary)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>(non)term.</i>	<i>meaning</i>
left	β	boolean	D	true if and only if nonterminal D generates a string that ends with letter b
left	χ	boolean	D	true if and only if nonterminal D generates a string that starts with letter c
left	e	boolean	D	true if and only if nonterminal D generates the empty string
left	ac	boolean	S, D	true at the tree root if and only if letter pair ac occurs in the Dyck string
left	bd	boolean	S, D	true at the tree root if and only if letter pair bd occurs in the Dyck string

attributes to be added, if any (complete the specification)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>(non)term.</i>	<i>meaning</i>

$$0: S_0 \rightarrow D_1$$

$$1: D_0 \rightarrow a D_1 b D_2$$

$$2: D_0 \rightarrow c D_1 d D_2$$

$$3: D_0 \rightarrow \varepsilon$$

$$0: S_0 \rightarrow D_1$$

$$1: D_0 \rightarrow a D_1 b D_2$$

$$2: D_0 \rightarrow c D_1 d D_2$$

$$3: D_0 \rightarrow \varepsilon$$

$$0: S_0 \rightarrow D_1$$

$$1: D_0 \rightarrow a D_1 b D_2$$

$$2: D_0 \rightarrow c D_1 d D_2$$

$$3: D_0 \rightarrow \varepsilon$$

